main.tex

FROM MLE TO FULL POSTERIOR INFERENCE IN PFNS

Theoretical Analysis and Limitations

```
$ make report
> compiling sections ........... ok
> computing posteriors ......... ok
```

// //

> ls report/

01 From MLE to Posterior Inference
02 Prior Distribution & Objective Function
03 Kullback-Leibler Divergence & Optimization
04 Limitations of the PFN Framework
05 From PPD Approximation to TabM

> cd section_01

01

FROM MLE TO POSTERIOR INFERENCE

MAXIMUM LIKELIHOOD ESTIMATION (MLE)

Consider a supervised learning problem on a dataset $D_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{n}$ and a query point x. The goal is to determine the predictive distribution $p(y \mid x, D_{\text{train}})$.

Traditional Machine Learning with MLE

The inference process is reduced to finding a single hypothesis $\hat{\phi}_{\text{MLE}}$ that maximizes the probability of generating the training data:

$$\hat{\phi}_{\text{MLE}} = \arg\max_{\phi \in \Phi} p(D_{\text{train}} \mid \phi) = \arg\max_{\phi \in \Phi} \sum_{i=1}^{n} \log p(y_i \mid x_i, \phi)$$

When predicting on a new point x, the MLE/MAP model solely relies on this point estimate:

$$p(y \mid x, D_{\text{train}}) \approx p(y \mid x, \hat{\phi}_{\text{MLE}})$$

Posterior Predictive Distribution

According to the law of total probability, the exact PPD must be marginalized by integrating over the entire hypothesis space Φ:

$$p(y \mid x, D_{\texttt{train}}) = \int_{\Phi} p(y \mid x, \phi)\, p(\phi \mid D_{\texttt{train}})\, d\phi$$

The Degeneracy of MLE/MAP: MLE applies the extreme assumption that the posterior distribution degenerates into a **Dirac delta function** at $\hat{\phi}$:

$$p_{\texttt{MLE/MAP}}(\phi \mid D_{\texttt{train}}) = \delta(\phi - \hat{\phi})$$

$$\implies \int_{\Phi} p(y \mid x, \phi)\, \delta(\phi - \hat{\phi})\, d\phi = p(y \mid x, \hat{\phi})$$

Ignoring the area under the posterior curve easily leads to overfitting.

> cd section_02

02

PRIORS & OBJECTIVE FUNCTION

To compute the PPD integral without expensive MCMC methods, Prior-Data Fitted Networks (PFNs) transform the Bayesian integration problem into deep learning via **amortized inference**.

> We define a prior over the data-generating mechanism space $p(\phi)$ (e.g., SCM, BNN).
> The dataset $D = D_{\text{train}} \cup \{(x, y)\}$ is generated by sampling $\phi \sim p(\phi)$, and then sampling from $p(D \mid \phi)$.

The marginal distribution of the dataset $p(D)$ is defined by:

$$p(D) = \mathbb{E}_{\phi \sim p(\phi)}\big[p(D \mid \phi)\big] = \int_{\Phi} p(D \mid \phi)\, p(\phi)\, d\phi$$

OBJECTIVE FUNCTION OF PFNS

PFNs use a Transformer network with parameters θ to model the output distribution $q_\theta(y \mid x, D_{\texttt{train}})$.

Optimization Process

Train the PFN by continuously sampling synthetic datasets $D \sim p(D)$ and minimizing the expected Cross-Entropy loss on the test point (x, y):

$$\mathcal{L}_{\texttt{PFN}}(\theta) = \mathbb{E}_{(x,y)\cup D_{\texttt{train}} \sim p(D)} \big[-\log q_\theta(y \mid x, D_{\texttt{train}}) \big]$$

> cd section_03

03

KL DIVERGENCE & OPTIMIZATION

By the chain rule $p(D) = p(x, D_{\text{train}})\, p(y \mid x, D_{\text{train}})$ and the law of iterated expectations, the loss function becomes:

$$\mathcal{L}_{\text{PFN}}(\theta) = \mathbb{E}_{p(x, D_{\text{train}})} \left[\mathbb{E}_{p(y|x, D_{\text{train}})} \left[-\log q_\theta(y \mid x, D_{\text{train}}) \right] \right]$$

Relationship between Entropy and KL Divergence

Using the identity connecting Cross-Entropy, Shannon Entropy $\mathcal{H}$, and KL Divergence:

$$\mathbb{E}_{y \sim p}[-\log q_\theta(y)] = D_{\text{KL}}\big(p \parallel q_\theta\big) + \mathcal{H}\big(p\big)$$

Since the Entropy $\mathcal{H}\big(p(\cdot \mid x, D_{\text{train}})\big)$ is independent of the network parameters θ, minimizing $\mathcal{L}_{\text{PFN}}$ is equivalent to minimizing the expected KL divergence.

GLOBAL OPTIMAL SOLUTION

The equivalent direct minimization problem:

$$\arg\min_{\theta} \mathbb{E}_{p(x, D_{\texttt{train}})} \left[D_{\mathsf{KL}} \big(p(\cdot \mid x, D_{\texttt{train}}) \parallel q_\theta(\cdot \mid x, D_{\texttt{train}}) \big) \right]$$

Convergence to the PPD

The network's optimal solution exactly approximates the Bayesian Posterior Predictive Distribution:

$$q_{\theta^*}(y \mid x, D_{\texttt{train}}) = p(y \mid x, D_{\texttt{train}}) = \int_{\Phi} p(y \mid x, \phi)\, p(\phi \mid D_{\texttt{train}})\, d\phi$$

> cd section_04

04

LIMITATIONS OF PFNs

LIMITATION 1: PRIOR SUPPORT AND INDUCTIVE BIAS

The optimal solution is only mathematically guaranteed in data regions where the prior density $p(D) > 0$.

- **Inductive Bias:** Combining SCMs and BNNs as priors imposes a specific bias. No single prior can cover all underlying rules.
- **Prior Misspecification:** Occurs when the real-world data is generated by a mechanism $\phi_{\text{real}} \notin \text{supp}(p(\phi))$.

LIMITATION 2: SCALING BOTTLENECKS ON LARGE DATA

The expectation $\mathbb{E}_{p(D)}$ being sampled over a fixed space creates barriers for scaling:

[1] Applying it to a dataset with $N > N_{\text{Training}}$ (with $O(N^2)$ complexity) causes the weight distribution to shift away from the initial training domain, breaking the posterior approximation.

[2] Bayesian asymptotics indicate that as $N \to \infty$, the data completely overwhelms the prior.

> cd section_05

05

FROM PPD APPROXIMATION TO TABM

PPD APPROXIMATION VIA DEEP ENSEMBLES

Since PFNs are limited by $O(N^2)$ complexity on large datasets, a more direct approach is needed to compute the PPD integral:

$$p(y \mid x, D_{\texttt{train}}) = \int_{\Phi} p(y \mid x, \phi)\, p(\phi \mid D_{\texttt{train}})\, d\phi$$

Monte Carlo Approximation via Deep Ensembles

Instead of using amortized inference on a synthetic prior like PFNs, combining k independent models $\{\phi_i\}_{i=1}^{k}$ trained with different random seeds acts as sampling over the posterior hypothesis space:

$$p(y \mid x, D_{\texttt{train}}) \approx \frac{1}{k} \sum_{i=1}^{k} p(y \mid x, \phi_i)$$

A Deep Ensemble's prediction is the average of its members' predictions, leveraging diverse information from the input data and significantly outperforming a single model.

MOTIVATION 1: LIMITATIONS OF INDEPENDENT TRAINING ($\text{MLP}^{\times k}$)

$\text{MLP}^{\times k}$ is a traditional Deep Ensemble: performing hyperparameter tuning for a single MLP, then independently training k tuned models under different random seeds.

Suboptimal for Ensemble Task Performance

> **Individual Early Stopping:** The training process of each MLP is halted based on its individual validation score. This is optimal for the individual MLP but can be suboptimal for the entire ensemble.

> **Isolated Hyperparameter Tuning:** Hyperparameters are only tuned for a single MLP without any awareness of the subsequent ensembling step.

Cost and Inconvenience

Independently training, storing, and deploying multiple models incurs high computational costs and practical inconvenience.

MOTIVATION 2: BENEFITS OF PARALLEL TRAINING ($\text{TabM}_{\text{packed}}$)

$\text{TabM}_{\text{packed}}$ packs k independent MLPs (without weight sharing) into a single model via Packed-Ensemble, enabling parallel processing of k inputs.

1. Collective Training & Ensemble-aware Stopping

> One training step is equivalent to k training steps of independent MLPs.

> Monitor the whole ensemble's performance during training and apply early stopping when the **entire ensemble** reaches its optimal state.

> Perform hyperparameter tuning on $\text{TabM}_{\text{packed}}$ as a unified model.

2. Runtime & Model Size Overhead

> **Runtime:**

Executes k forward passes in parallel on modern hardware, thus the runtime overhead increases **significantly less than** $\times k$.

> **Model Size:**

Parameter memory footprint still scales linearly ($\times k$) due to the lack of weight sharing.

To reduce the footprint of $\text{TabM}_{\text{packed}}$, **TabM** applies a weight sharing mechanism among implicit submodels based on BatchEnsemble:

Parameterizing Linear Layers with Adapters

Given a shared weight matrix $W \in \mathbb{R}^{d \times d}$, the i-th submodel is characterized by non-shared adapters $r_i, s_i, b_i \in \mathbb{R}^d$:

$$l_i(x_i) = s_i \odot \big(W(r_i \odot x_i)\big) + b_i \iff W_i = W \odot (s_i r_i^\top)$$

In matrix form for $X \in \mathbb{R}^{k \times d}$ and adapters $R, S, B \in \mathbb{R}^{k \times d}$:

$$l_{\text{BE}}(X) = \big((X \odot R)W\big) \odot S + B$$

> **Negligible Model Size Overhead:** Adding a submodel only requires appending 1 row to R, S, B, introducing exactly $3d$ new parameters per layer — negligible compared to the original $d^2 + d$ parameters.

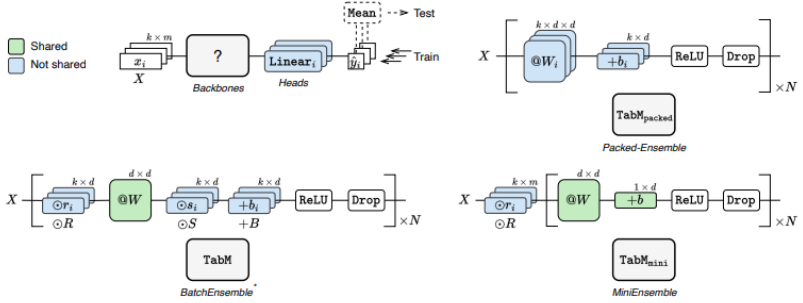


Figure 1: *(Upper left)* A high-level illustration of TabM. One TabM represents an ensemble of k MLPs processing k inputs in parallel. The remaining parts of the figure are three different parametrizations of the k MLP backbones. *(Upper right)* $\text{TabM}_{\text{packed}}$ consists of k fully independent MLPs. *(Lower left)* TabM is obtained by injecting three non-shared adapters R, S, B in each of the N linear layers of *one* MLP (* the initialization differs from Wen et al. (2020)). *(Lower right)* $\text{TabM}_{\text{mini}}$ is obtained by keeping only the very first adapter R of TabM and removing the remaining $3N - 1$ adapters. *(Details)* Input transformations such as one-hot-encoding or feature embeddings (Gorishniy et al., 2022) are omitted for simplicity. `Drop` denotes dropout (Srivastava et al., 2014).

EXPERIMENTS ON THE FIRST ADAPTER R AND INITIALIZATION

In an architecture with N blocks ($3N$ adapters), the authors demonstrate the critical role of the first adapter R at the very first linear layer:

Authors' Explanation

The first adapter R acts as a mapping from k inputs in the same representation space into k **distinct representation spaces** right before the tabular features are mixed for the first time via the $@W$ multiplication.

Ablation Study Results (compared to baseline MLP):

- **Removing the first R (TabM$_{\text{bad}}$):** Deleting the first R while keeping the other $3N-1$ adapters $\implies$ Performance drops significantly.
- **Keeping only the first R (TabM$_{\text{mini}}$):** Retaining solely the first R and discarding all other $3N-1$ adapters $\implies$ Outperforms even TabM$_{\text{naive}}$.
- **Better Initialization (TabM):** Keeping all $3N$ adapters, randomly initializing the first R with ± 1, and deterministically initializing all other R, S with 1 (initially behaving like TabM$_{\text{mini}}$ but free to increase expressivity during training) $\implies$ Best performance.

1. Weak Individually, Powerful Collectively

Analysis of the training profiles for $\text{TabM}^{k=32}_{\text{mini}}$ reveals: individual implicit submodels are heavily **overfitted** compared to a single MLP (even the best submodel $\text{TabM}[B]$ is no better than a plain MLP). However, thanks to the genuine diversity among the submodels, their collective mean prediction achieves superior generalization.

2. Deep Ensembling TabM Itself

> Traditional Ensembling: Averaging 5 independently trained $\text{TabM}_{\text{mini}}$ models ($\text{TabM}^{\times 5}_{\text{mini}}$) further pushes the average performance improvement.